INDIC-LLMLINGUA

Query-Aware Prompt Compression for Hindi Retrieval-Augmented Generation

Final Project Report

Data Science & Artificial Intelligence Project
Group 9
2026

Abstract

Large Language Models (LLMs) used in Retrieval-Augmented Generation (RAG) systems often receive long retrieved contexts, increasing inference cost and latency while making it harder for the generator to focus on query-relevant information. This project, Indic-LLMLingua, investigates query-aware prompt compression for Hindi RAG. The approach reformulates prompt compression as binary token classification: given a question and context, the model predicts whether each context token should be retained or removed. A teacher LLM is first used to distill query-relevant context, and the resulting compressed text is aligned with the original context to create token-level supervision. Hindi IndicQA data is combined with additional supervision generated from Hindi XL-Sum articles, producing an extended corpus for training. A multilingual XLM-RoBERTa encoder with a linear token-classification head is fine-tuned using cross-entropy-based token supervision, subword ignore masking, dynamic padding, mixed precision, gradient checkpointing, and validation-F1-based checkpoint selection. Initial hyperparameter experiments identify a learning rate of 2×10^{-5}, batch size 8, weight decay 0.01, and 100 warmup steps as a strong configuration. A subsequent 20-epoch training run reports a best validation F1 of 0.59, with evaluation-set accuracy of 0.73, precision of 0.53, recall of 0.67, and F1 of 0.59. Additional analysis reports ROC-AUC of approximately 0.83. Qualitative analysis demonstrates substantial context reduction while retaining query-relevant entities and facts, although errors remain concentrated around difficult and low-frequency tokens. The results establish a working Hindi query-aware token compressor and identify clear directions for improving precision, recall balance, and downstream RAG evaluation.

Keywords

Prompt compression; Retrieval-Augmented Generation; Hindi NLP; IndicQA; XLM-RoBERTa; token classification; query-aware compression; dataset distillation.

Table of Contents

Introduction

Background

Retrieval-Augmented Generation combines information retrieval with generative language models by supplying retrieved documents or passages as additional context. Although this design improves the availability of external knowledge, retrieved context can become substantially longer than the information actually needed to answer a user's question. Processing such context increases the number of input tokens consumed by the generator and can increase inference cost and latency. Long prompts may also make it more difficult for a model to identify the specific evidence required for an answer.

Prompt compression addresses this problem by removing redundant or low-value portions of the context before the compressed prompt is passed to the downstream language model. Existing approaches demonstrate that prompt compression can reduce the effective context length, but the suitability of these methods for regional and morphologically rich languages remains an important question. The present project focuses on Hindi, where preserving word forms, grammatical endings, and semantic relationships is particularly important.

Problem Statement

The project addresses two limitations identified in existing prompt compression systems. First, English-centric compressors can cause morphological destruction when transferred to Hindi, including arbitrary fragmentation or removal of tokens that are linguistically important. Second, task-agnostic token classifiers may assign preservation scores without directly considering the user's question. In a RAG setting, this can result in the removal of facts that are specifically required to answer the query.

Our model therefore formulates compression as a query-aware token classification problem. The model receives both the question and context and predicts a binary decision for each context token: 1 for KEEP and 0 for REMOVE. The desired behavior is to remove redundant context while retaining the information required for question answering.

Research Gap

LLMLingua and LongLLMLingua demonstrate the potential of learned prompt compression, while LLMLingua-2 makes compression more efficient by formulating it as token classification. However, the project literature review identifies a gap for regional

RAG applications (especially Hindi): an efficient token-classification compressor should also account explicitly for the query and should be trained on language-specific data. Our Model addresses this gap through query-conditioned inputs and Hindi-specific supervision generated from IndicQA and XL-Sum.

Objectives

1. Develop a Hindi-focused query-aware prompt compression model using token classification.
2. Generate reliable token-level supervision by distilling contexts with a teacher LLM and aligning the compressed text back to the original context.
3. Train a multilingual Transformer encoder to identify context tokens that should be retained or removed.
4. Optimize the model using validation F1 as the primary model-selection metric because the token labels are highly imbalanced.
5. Evaluate the resulting model quantitatively and qualitatively, including precision, recall, F1, accuracy, ROC-AUC, average precision, error patterns, and compression examples.
6. Establish a foundation for subsequent downstream RAG experiments comparing compressed and uncompressed contexts.

Scope and Boundaries

The project focuses on **Hindi** and uses a smaller **XLM-RoBERTa-base** model to efficiently test and validate the proposed approach before experimenting with larger models. The training dataset contains approximately **2,000 samples from IndicQA and XL-Sum**. Further improvements could be achieved using a larger dataset and **focal loss or other class-imbalance techniques** to improve KEEP-token prediction.

Literature review

- LLMLingua & LongLLMLingua (Jiang et al., 2023): These approaches utilize causal small language models (SLMs) to compress prompts based on information entropy and perplexity. While LongLLMLingua is task-aware, its autoregressive generation process is computationally heavy and slow.
- LLMLingua-2 (Pan et al., 2024): This framework treats prompt compression as a token classification task (preserve/discard) using a bidirectional Transformer

encoder. It captures full context and is highly efficient (achieving 1.6x-2.9x end-to-end speedups), but the model is entirely task-agnostic and trained exclusively on English meeting transcripts (MeetingBank).
- IndicQA Benchmark (2024): A recent comprehensive benchmark for non-factoid Question Answering in 11 Indic languages. It highlights the complexities of retrieving and reasoning over regional texts but does not inherently provide a latency-optimized prompt compression framework.

Dataset and Methodology

Data Source

We used the Hindi subset of the IndicQA dataset as the primary source for our experiments. The dataset provides Hindi questions, corresponding contexts, and ground-truth answers. From the initial **1,052 Hindi QA samples**, 987 samples were successfully processed through our teacher-based distillation pipeline. The remaining samples were skipped because the teacher model returned null outputs.

We further expanded the training data using the **Hindi subset of the XL-Sum dataset**, resulting in a combined dataset of approximately **2,000 samples** for training and evaluation.

This version makes it clear that **IndicQA and XL-Sum are the project's data sources**, while the distillation and dataset construction are our project methodology.

Data Organization and Reproducibility

The project separates raw and processed data and maintains dedicated scripts for acquisition, distillation, validation, and model training. The principal data artifacts include raw_hindi_qa.json, train/validation/test JSONL files, and preprocessing and training notebooks. This separation makes it possible to rerun individual stages without mixing raw inputs with generated supervision.

Query-Aware Teacher Distillation

To convert QA examples into compression supervision, each question-context pair is sent to a teacher LLM. The teacher is instructed to remove information irrelevant to the question while preserving all information needed to answer it. The prompt explicitly prohibits grammar changes, translation, paraphrasing, and introduction of new words. A low temperature of 0.1 is used to make the compression behavior more deterministic.

The teacher output is not directly used as the final compressor. Instead, it acts as a supervision generator. This is a form of dataset distillation: a generative model creates a compressed version of the context, and that output is subsequently converted into token-level labels for supervised training.

XL-sum Data Expansion

Milestone 4 extends the original IndicQA supervision with additional Hindi examples generated from the XL-Sum dataset. The expansion follows three stages: (1) generate a question-answer pair whose answer is inferable from the article, (2) compress the article under a question-answer preservation constraint, and (3) align the compressed text to the original article to create binary token labels.

Figure 1. End-to-end query-aware prompt compression workflow

Token Alignment and Label Generation

Let the original whitespace-tokenized context be $T = [t_1, t_2, \ldots, t_n]$. The compressed teacher output is tokenized into a set S of preserved tokens. Each original token receives a binary label:

$$y_i = 1 \text{ if } t_i \in S; \text{ otherwise } y_i = 0.$$

This converts the original generative compression task into a sequence-labeling task. Label 1 means KEEP and label 0 means REMOVE. The original word order is preserved when the model later reconstructs the compressed context.

Whitespace matching introduces potential issues when punctuation is attached to words or when the teacher output changes surface forms. The project therefore uses a subword-aware alignment stage during model preprocessing and applies an ignore index of -100 to question tokens, continuation subwords, padding, and special tokens so that loss and evaluation are calculated only at valid context positions.

Input Representation

The model receives both the question and context so that token importance is conditioned on the query. XLM-RoBERTa's SentencePiece tokenizer converts the combined text into multilingual subword units. The conceptual structure is Question + Context, while the model tokenizer inserts the appropriate special separators. Labels are aligned only to valid context subwords; non-target positions receive the ignore label.

Model Architecture

The implemented model uses FacebookAI/xlm-roberta-base as the feature encoder with a linear token-classification head. The encoder contains 12 Transformer layers, a 768-dimensional hidden representation, and 12 attention heads. The classification head maps each contextual token representation from 768 dimensions to two logits corresponding to REMOVE and KEEP.

Component	Configuration
Encoder	FacebookAI/xlm-roberta-base
Parameters	277,454,594 (~277.45M)
Transformer layers	12
Hidden size	768
Attention heads	12
Classification head	Linear: 768 → 2
Output labels	0 = REMOVE; 1 = KEEP
Maximum sequence length	512 tokens

Training Objective and Stability Measures

The training objective is token-level classification. The predicted logits are compared with the binary labels using cross-entropy-based supervision, while ignored positions do not contribute to the loss. The use of -100 for continuation subwords is particularly important because Hindi words can be split into multiple subword units; counting every continuation as an independent supervised target could distort the training signal.

- AdamW optimization with weight decay.
- Linear learning-rate warmup followed by decay.
- FP16 mixed precision to reduce memory usage and accelerate computation.
- Gradient checkpointing to reduce GPU memory consumption.
- Dynamic padding through DataCollatorForTokenClassification.
- Validation F1 used for best-checkpoint selection instead of accuracy.

Model Development and Hyperparameter Tunning

Baseline Training Configuration

The initial full training experiment used a learning rate of 2×10^{-5}, per-device batch size of 8, gradient accumulation of 2 steps, weight decay of 0.01, 100 warmup steps, maximum sequence length of 512, FP16 precision, and gradient checkpointing. Training was conducted on dual NVIDIA Tesla T4 GPUs in a Kaggle/Linux environment.

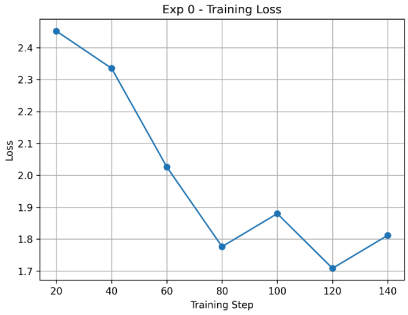
Hyperparameter Experiments

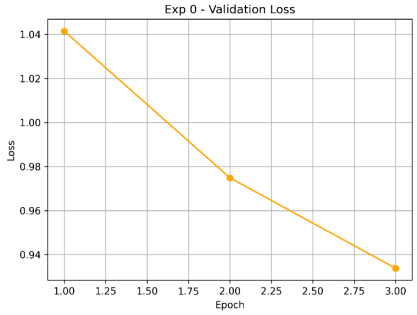
The performance of the XLM-RoBERTa token classification model was evaluated after training under different hyperparameter configurations. Each experiment was conducted independently using the same training and validation datasets while varying selected hyperparameters. The validation set was used to compare different configurations and identify the model that achieved the best overall performance.

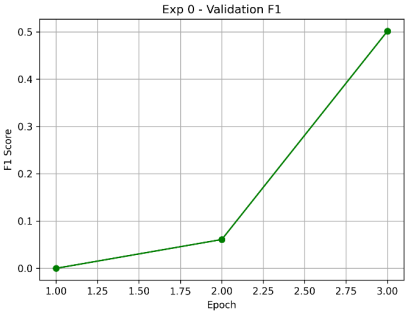
Model performance was evaluated using Accuracy, Precision, Recall, and F1-score, with the F1-score serving as the primary criterion for selecting the best-performing configuration. The notebook also records the training loss and validation loss throughout the training process, enabling analysis of convergence behaviour and model generalization.

Experiment	Learning rate	per_device_train_batch_size	weight_decay	Warmup steps	Validation Accuracy	Validation Precision	Validation Recall	Validation F1
0	1e-05	8	0.01	100	0.73	0.54	0.46	0.50
1	2e-05	8	0.01	100	0.74	0.55	0.53	0.54
2	3e-05	16	0.01	200	0.72	0.51	0.36	0.42
3	2e-05	8	0.1	100	0.74	0.57	0.42	0.48

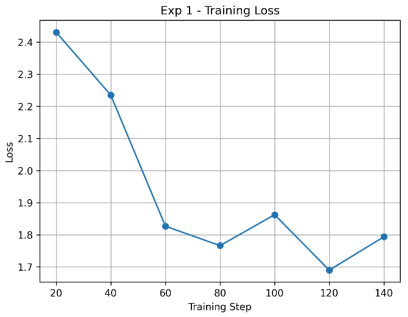
Experiment 0: {'learning_rate': 1e-05, 'per_device_train_batch_size': 8, 'weight_decay': 0.01, 'warmup_steps': 100}

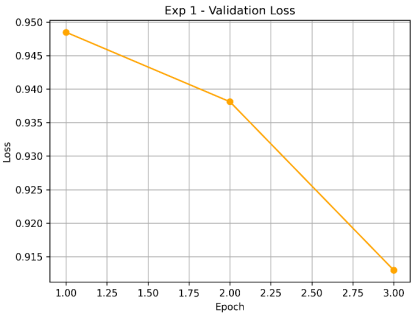


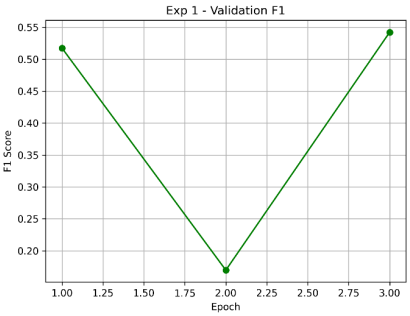




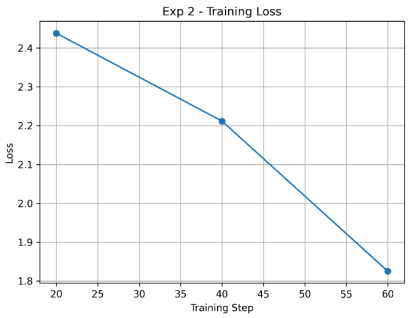
Experiment 1: {'learning_rate': 2e-05, 'per_device_train_batch_size': 8, 'weight_decay': 0.01, 'warmup_steps': 100}

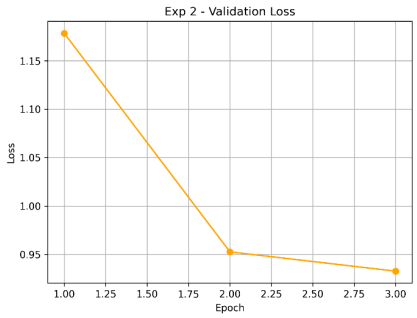


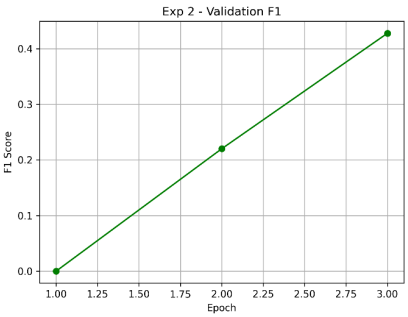




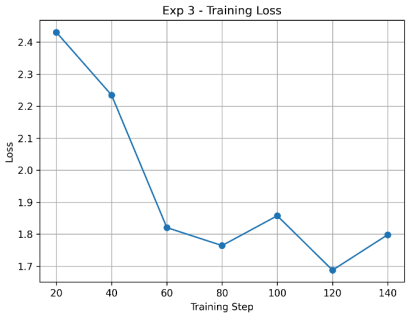
Experiment 2: {'learning_rate': 3e-05, 'per_device_train_batch_size': 16, 'weight_decay': 0.01, 'warmup_steps': 200}

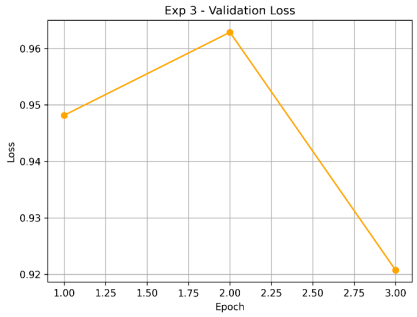


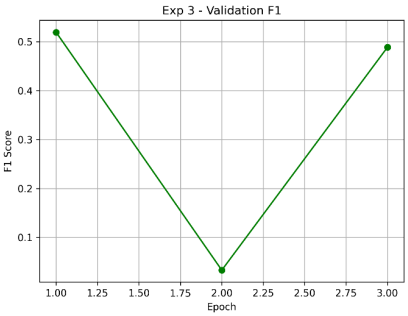


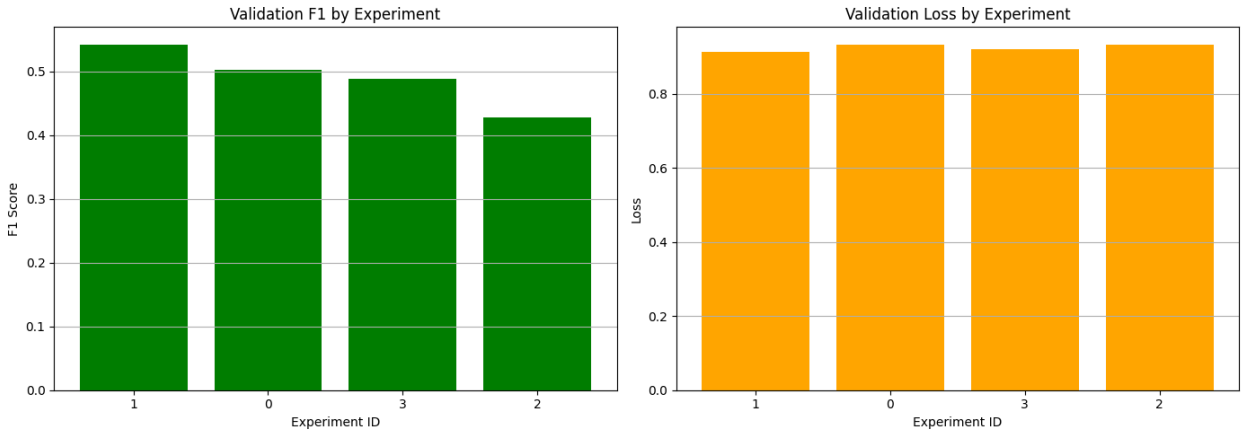


Experiment 3: {'learning_rate': 2e-05, 'per_device_train_batch_size': 8, 'weight_decay': 0.1, 'warmup_steps': 100}







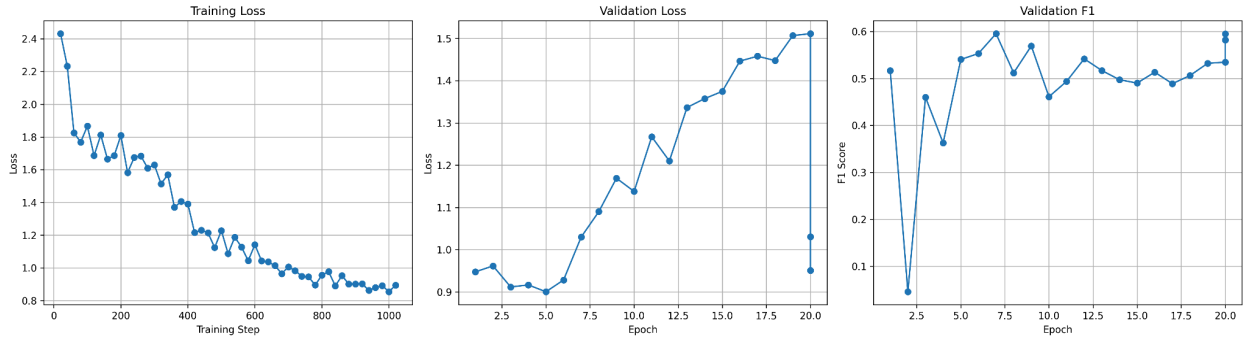


The experimental results show that all configurations achieved a reduction in training and validation loss across epochs, indicating successful model learning and convergence. Among the four experiments, **Experiment 1**(i.e exp id =1) achieved the highest validation F1-score (approximately **0.54**) while maintaining a low validation loss, making it the best-performing configuration overall. Although Experiments 0 and 3 produced comparable validation losses, their F1-scores were slightly lower, indicating that lower loss alone did not necessarily translate into better token classification performance. Experiment 2 recorded the lowest validation F1-score (approximately **0.43**) despite a similar validation loss, suggesting that its hyperparameter configuration was less effective at balancing precision and recall. Overall, the comparative analysis indicates that the hyperparameter settings used in **Experiment 1** provide the best trade-off between convergence and predictive performance, and were therefore selected as the optimal configuration for the final model.

Training Summary (Best model selected)

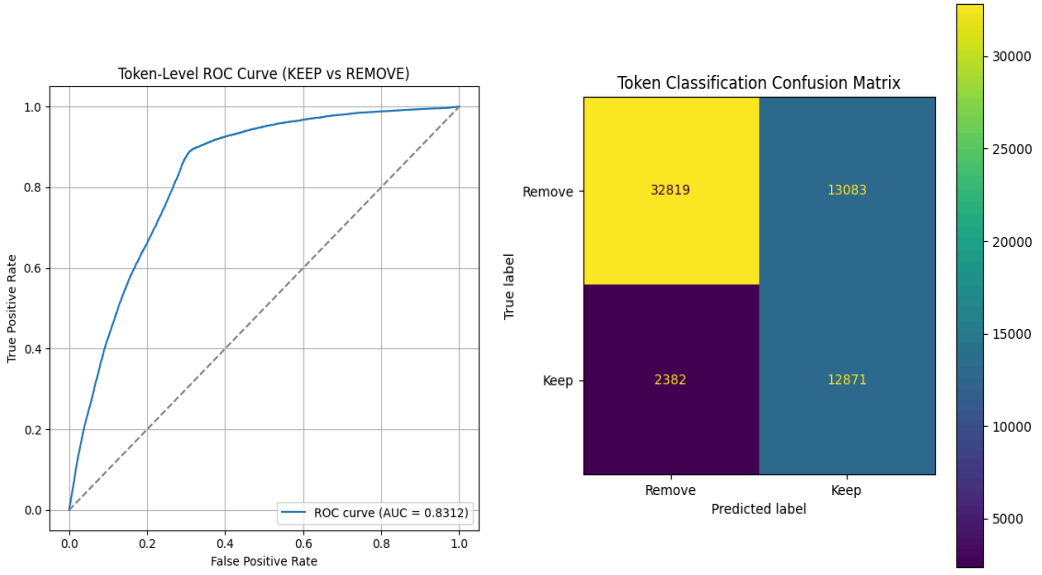
Parameter	Value
Learning Rate	2e-05
Batch Size	8
Weight decay	0.01
Warmup steps	100
Number of epochs	20
Best Validation F1	0.59

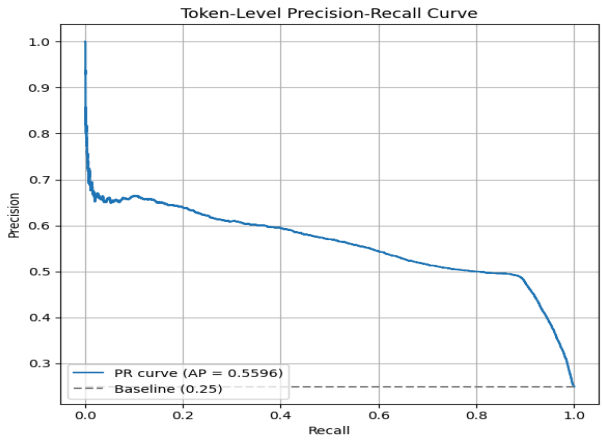
Final Model Performance on Evaluation set‘



Metric	Score
Loss	1.03
Accuracy	0.73
Precision	0.53
Recall	0.67
F1-score	0.59

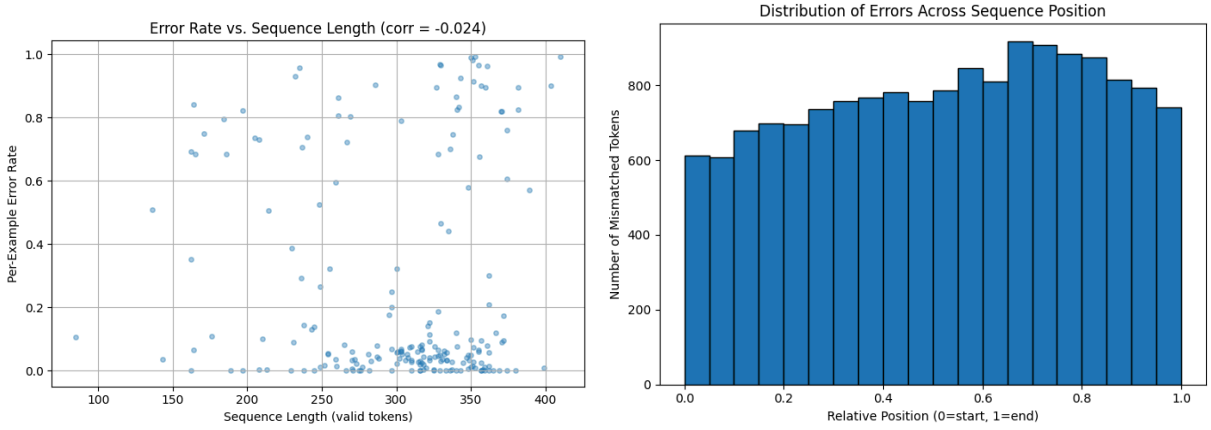
Evaluation and Analysis





The confusion matrix shows that the model correctly classifies a large proportion of both Remove and Keep tokens, with relatively fewer misclassifications, indicating good token-level prediction performance. The ROC curve achieves an AUC of approximately 0.83, demonstrating strong discrimination between the two classes across different decision thresholds. The Precision–Recall curve reports an Average Precision (AP) of approximately 0.60, indicating that the model maintains a reasonable balance between precision and recall for the positive (Keep) class. The gradual decline in precision as recall increases suggests the expected trade-off when identifying more relevant tokens. Overall, these evaluation plots indicate that the model generalizes well and provides reliable performance for the binary token classification task, with good discriminative ability and balanced predictive accuracy

Qualitative Results: Successful Predictions and Failure Cases



Token	Errors	Occurrences	Error Rate
_सोना	8	8	100.00%
_नित्य	7	7	100.00%
_सेक्स	6	6	100.00%
_देखी	6	6	100.00%
_विश्वकप	6	6	100.00%
_बनाकर	5	5	100.00%
_वित्तीय	5	5	100.00%
_टीवी	14	15	93.33%
_फ़िल्म	19	21	90.48%
_माँ	6	7	85.71%
_गेंद	6	7	85.71%
_चिंता	6	7	85.71%
_जाँच	10	12	83.33%
_पारी	5	6	83.33%
_अरुण	5	6	83.33%
_गिरावट	5	6	83.33%
_चट	5	6	83.33%
_नाका	5	6	83.33%
_संवाददाता	9	11	81.82%
_ठीक	9	11	81.82%

The qualitative analysis indicates that the model performs well on the majority of evaluation samples, correctly identifying **Keep** and **Remove** tokens, as reflected by the low error rates observed for most sequences. The error rate versus sequence length plot shows a **correlation of -0.024**, indicating that prediction errors are not strongly influenced by the input sequence length. This suggests that the model maintains relatively consistent performance across both shorter and longer sequences, with most samples exhibiting low error rates despite variations in sequence length.

The analysis of errors across sequence positions reveals that misclassifications occur throughout the sequence but are slightly more concentrated in the latter half (approximately **60%–90%** of the relative sequence length). This indicates that the model is generally robust across the input but encounters greater difficulty when predicting tokens appearing toward the end of longer contexts.

The token-level error analysis highlights several failure cases. Tokens such as **"_सोना", "_नित्य", "_सेक्स", "_देखी", "_विवेककुम", "_बनाकर", and "_वित्तीय"** exhibit an **error rate of 100%**, meaning they were misclassified in every occurrence within the evaluation set. Other challenging tokens include **"_टीवी" (93.33%)**, **"_फ़िल्म" (90.48%)**, **"_माँ" (85.71%)**, **"_गेंद" (85.71%)**, and **"_पिता" (85.71%)**, indicating that these specific words consistently posed difficulties for the model. These results suggest that the remaining prediction errors are concentrated around a small subset of tokens rather than being uniformly distributed across the dataset, providing clear targets for future improvements in data coverage and model refinement.

Model Limitations

The evaluation results indicate that the model still struggles with predicting certain tokens consistently. Token-level error analysis shows that several low-frequency tokens, such as **"_सोना", "_नित्य", "_सेक्स", "_देखी", "_विवेककुम", "_बनाकर", and "_वित्तीय"**, were misclassified in all of their occurrences, resulting in a **100% error rate**. Additionally, tokens such as **"_टीवी" (93.33%)**, **"_फ़िल्म" (90.48%)**, and **"_माँ" (85.71%)** also exhibited high error rates, suggesting that the model has difficulty generalizing to specific lexical patterns. The

overall **Validation F1-score of approximately 0.53** further indicates that there is room for improvement in balancing precision and recall for the binary token classification task.

Observed Anomalies

The evaluation revealed that prediction errors are **not correlated with input sequence length**, as indicated by the correlation coefficient of **-0.024** between sequence length and error rate. This suggests that increasing sequence length does not significantly degrade model performance. However, the distribution of errors across sequence positions shows that misclassifications are slightly more concentrated in the **latter portion of the sequence (approximately 60%–90% of the relative sequence length)**, indicating that the model finds later contextual tokens comparatively more challenging. Furthermore, although multiple experiments achieved nearly identical validation losses, their validation F1-scores differed noticeably, demonstrating that lower loss did not always correspond to better token classification performance.

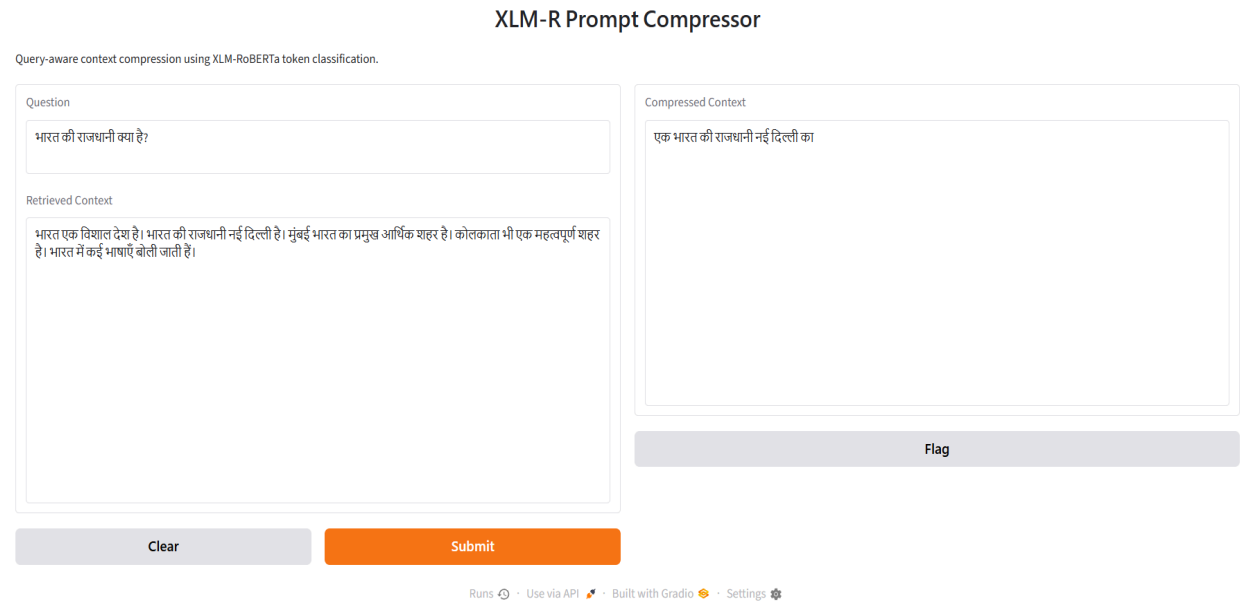
Gap Between Expected and Actual Performance

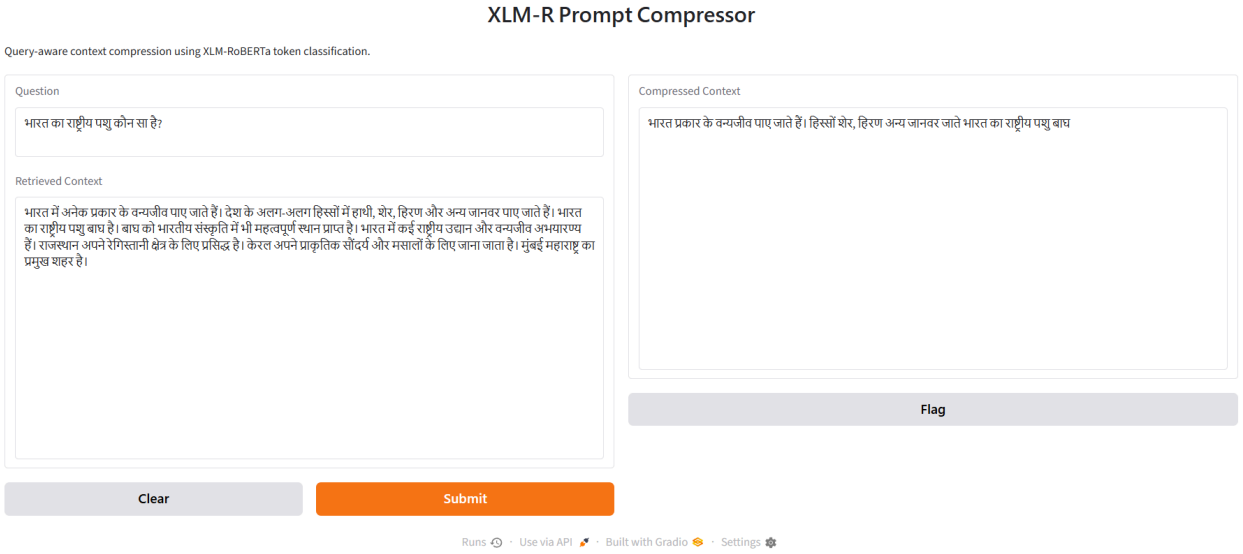
The objective of the model was to accurately identify **Keep** and **Remove** tokens with high precision and recall across all token types. While the model achieved a **ROC-AUC of approximately 0.83**, demonstrating good discriminative capability and **Validation F1-score of approximately 0.53** indicate that prediction quality remains moderate. The confusion matrix also shows that although most tokens are classified correctly, a noticeable number of **false positives and false negatives** are still present. These findings indicate that the model performs well on the majority of samples but requires further improvements to better handle difficult token categories and achieve more balanced token-level predictions.

Deployment

The trained model has been deployed on **Hugging Face Spaces** to provide an interactive interface for testing the token compression model. The deployed application is available as **TokenCompressor**: (https://huggingface.co/spaces/nnnhitesh/TokenCompressor)

The deployed interface allows users to provide a question and context and observe the resulting compressed context. Some sample outputs from the deployed application are presented below.





Additionally we had integrated the model with a rag pipeline to show the effective use case. In the rag pipeline we fetch data from chunks of document and before passing prompt to final model we pass question and fetched text to our data model for token compression to reduce token usage

Conclusion and Future Works

This project developed Indic-LLMLingua, a query-aware prompt compression framework designed for Hindi RAG. The central design choice was to replace query-independent or generative compression with a token-classification formulation in which the question and context are jointly encoded. Teacher-generated compressed contexts were aligned with their originals to create binary KEEP/REMOVE supervision, and additional Hindi examples were generated from XL-Sum to expand the training corpus.

The experiments demonstrate that the approach can learn useful token-selection behavior. The selected configuration uses a learning rate of 2×10^{-5}, batch size 8, weight decay 0.01, and 100 warmup steps, and achieves a best validation F1 of 0.59 in the reported 20-epoch training run. The evaluation set achieves accuracy 0.73, precision 0.53, recall 0.67, and F1 0.59, with ROC-AUC around 0.83 and average precision around 0.60. Qualitative results show that the model can remove substantial context while retaining important query-related information.

At the same time, the results reveal that compression quality is constrained by the precision-recall trade-off and by difficult lexical cases. The model is conservative enough to preserve many relevant tokens, but this also causes unnecessary tokens to remain. The observed errors on low-frequency Hindi tokens further indicate that more representative training data and improved alignment could be beneficial.

Future Works

1. Evaluate downstream RAG answer quality using Exact Match and F1 by comparing full-context and compressed-context prompts.
2. Measure end-to-end RAG latency and token usage to determine whether the compressor produces a practical speedup after its own inference cost is included.
3. A larger model could be trained for better performance. We had limited ourselves to comparatively smaller models.
4. Prompt compression technique could be extended to more Indic languages like Bengali, Tamil, Marathi etc.
5. Experiment with stronger class-imbalance strategies and regularization to improve precision without sacrificing recall.